시스템프로그래밍

Assignment1-1

제출일자: 2026-03-30

담당 교수님: 최상호

학번: 2022202094

이름: 이원우

분반: 금 1,2 (B)

Introduction

본 과제는 getopt() 함수를 활용하여 명령 인자를 분석하는 프로그램을 구현하는 것을 목표로 한다. 사용자가 입력한 옵션 -a, -b, -c 를 파싱하고, 각 옵션의 동작을 프로그램 내부 변수에 반영하는 구조를 설계하였다. 특히 getopt()를 반복문 내에서 호출하여 옵션을 순차적으로 처리하고, 옵션 처리 이후 남은 일반 인자(non-option argument)를 별도로 처리하는 방식으로 구현하였다.

또한 옵션의 조합 형태(-ab, -cfoo)와 반복 입력(-aa) 등 다양한 입력 상황을 고려하여 프로그램이 일관된 결과를 출력하도록 구성하였다. 이를 통해 명령행 인터페이스 프로그램의 기본적인 입력 처리 구조를 이해하는 것을 목적으로 한다.

결과 화면

터미널 화면

```
kw2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -aa
aflag = 2, bflag = 0, cvalue = (null)
kw2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -d-a
aflag = 1, bflag = 0, cvalue = (null)
kw2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt foo foo
aflag = 0, bflag = 0, cvalue = (null)
Non-option argument foo
Non-option argument foo
```

```
2022202094@ubuntu:~/Downloads$ gcc kw2022202094_opt.c -o kw2022202094_opt
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt
lag = 0, bflag = 0, cvalue = (null)
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -a -b
lag = 1, bflag = 1, cvalue = (null)
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -ab
lag = 1, bflag = 1, cvalue = (null)
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -c foo
lag = 0, bflag = 0, cvalue = foo
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -cfoo
lag = 0, bflag = 0, cvalue = foo
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt arg1
lag = 0, bflag = 0, cvalue = (null)
n-option argument arg1
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -a arg1
lag = 1, bflag = 0, cvalue = (null)
n-option argument arg1
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -c foo arg1
lag = 0, bflag = 0, cvalue = foo
n-option argument arg1
2022202094@ubuntu:~/Downloads$ ./kw2022202094_opt -a -
lag = 1, bflag = 0, cvalue = (null)
n-option argument -
```

먼저 프로그램 실행 시, -a, -b, -c 옵션이 입력되었는지를 판단하여 각각의 결과를 출력한다. -a와 -b 옵션은 등장 횟수를 기준으로 카운트되며, 반복 입력 시 그 횟수가 누적되어 출력된다. 예를 들어 -aa 입력 시 aflag 값이 2로 증가하는 것을 통해 동일 옵션의 반복 처리가 가능함을 확인할 수 있다.

-c 옵션의 경우 문자열 인자를 필요로 하며, 입력된 문자열은 cvalue 변수에 저장되어 그대로 출력된다. 이때 -c foo와 -cfoo 두 가지 입력 방식이 모두 동일하게 처리되며, 이는 getopt()가 옵션과 인자를 유연하게 해석함을 보여준다. 반면 -c 옵션이 입력되지 않은 경우에는 cvalue가 설정되지 않아 (null)로 출력된다.

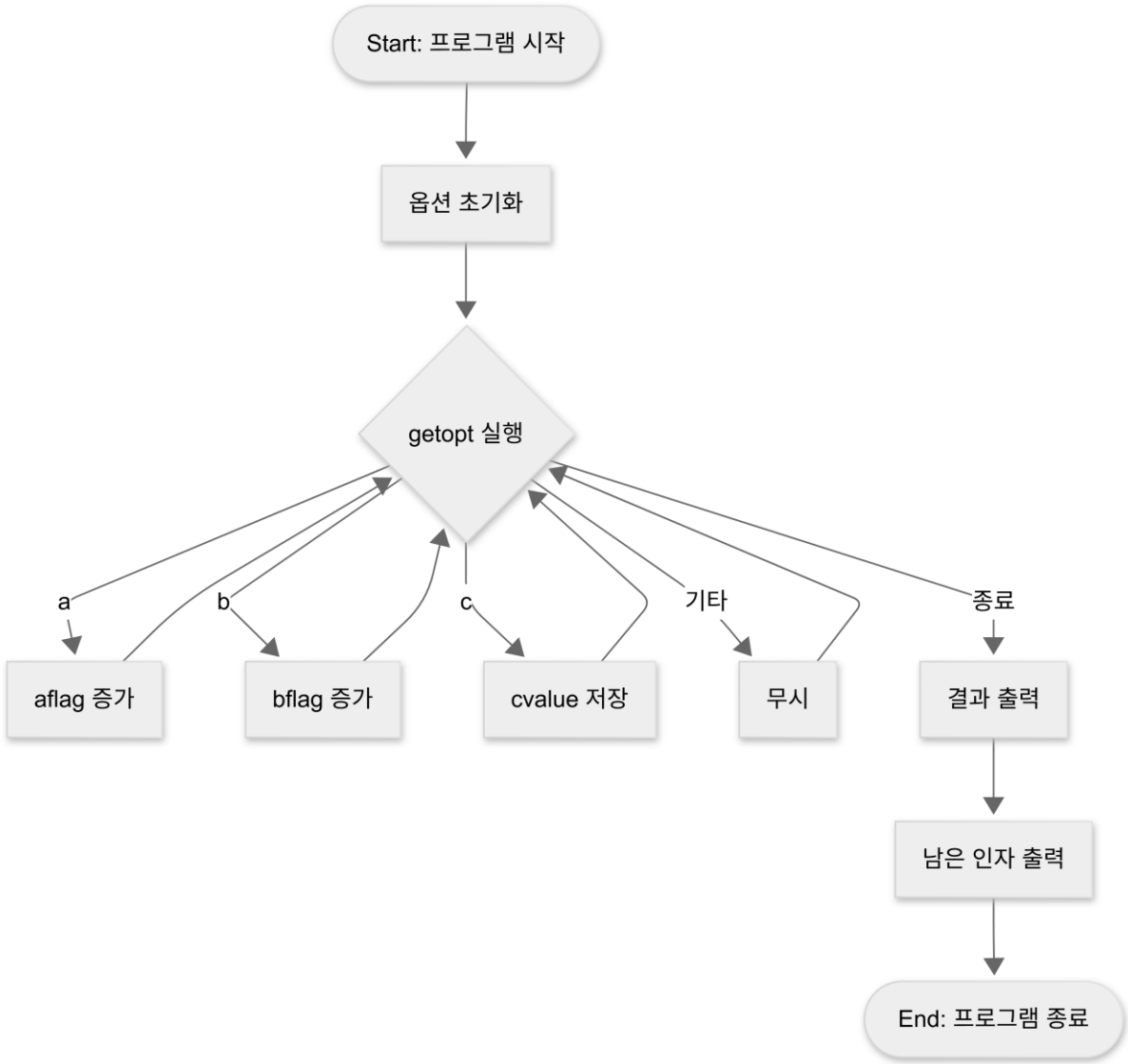
옵션 처리가 종료된 이후에는 getopt()가 더 이상 처리할 옵션이 없음을 판단하고 반복문이 종료된다. 이때 내부 인덱스(optind)는 남아 있는 첫 번째 인자를 가리키며, 이후 별도의 반복문을 통해 해당 위치부터 끝까지의 인자를 순차적으로 출력한다. 이 과정에서 -로 시작하지 않는 일반 문자열 인자들은 Non-option argument로 분류되어 출력된다.

또한 정의되지 않은 옵션(예: -d)이 입력된 경우에도 프로그램은 오류를 출력하지 않고 무시하며, 이후 정상적인 옵션만 처리한다. 이는 opterr = 0 설정에 의해 getopt()의 자동 에러 메시지를 비활성화한 결과이다.

결과적으로 본 프로그램은 다양한 입력 형태(단일 옵션, 결합 옵션, 옵션 인자 포함, 반복 옵션, 일반 인자 포함)에 대해 일관된 출력 결과를 제공하며, 명령행 인터페이스에서 옵션과 일반 인자를 구분하여 처리하는 기본적인 구조를 확인할 수 있다.

Algorithm

Flow Chart



Pseudo Code

```
START

initialize aflag = 0, bflag = 0
initialize cvalue = NULL
disable getopt error messages

WHILE (there is an option using getopt)
    IF option is 'a'
        increment aflag
    ELSE IF option is 'b'
        increment bflag
    ELSE IF option is 'c'
        store argument in cvalue
    ELSE
        ignore
END WHILE

print aflag, bflag, cvalue

set index = optind

WHILE (there are remaining arguments)
    print non-option argument
    move to next argument
END WHILE

END
```

고찰

본 실습에서는 지난 Ubuntu 환경에서 진행했던 Make 및 Linux 기반 개발 경험을 바탕으로, getopt() 함수를 활용하여 명령행 인자를 분석하는 프로그램을 구현하였다. 별도의 실습 과목을 수강하지 않은 상태에서 진행되었기 때문에, unistd 내부 함수와 관련 동작을 직접 찾아가며 이해하고 적용해야 했다.

초기에는 getopt()의 동작 방식과 optind, optarg와 같은 내부 변수의 의미를 정확히 파악하는 데 어려움이 있었으나, 반복적인 실험과 코드 수정 과정을 통해 옵션이 하나씩 처리되는 구조와 인자 소비 방식에 대한 이해를 점차 확립할 수 있었다. 특히 옵션과 일반 인자를 구분하여 처리하는 구조를 구현하는 과정에서, CLI 기반 프로그램의 기본적인 입력 처리 흐름을 체계적으로 이해할 수 있었다.

또한 기존에 사용하던 C/C++ 기반 프로그래밍 방식과 크게 다르지 않다는 점에서 비교적 빠르게 적응할 수 있었으며, 직접 Linux 환경에서 컴파일 및 실행을 반복하면서 개발 과정 전반에 대한 감각을 익힐 수 있었다. 이러한 과정을 통해 Linux 기반 개발 환경에 점차 익숙해지고 있다는 점을 체감할 수 있었으며, 향후 보다 복잡한 시스템 프로그래밍 과제를 수행하는 데 있어 기초적인 기반을 마련할 수 있었다.